

BMCS2063 Data Structures and Algorithms

Sorted Lists

Chapter 7

Introduction

- For the ADT list, entries are ordered simply by their *positions*.
- However, some applications require *sorted data*.
 - Hence, an ADT that maintains data in sorted order would be convenient.

Learning Outcomes

At the end of this lecture, you should be able to

- Use a sorted list in a program
- Describe the differences between the ADT list and the ADT sorted list
- Implement the ADT sorted list by using an array
- Implement the ADT sorted list by using a chain of linked nodes

Specifications for the ADT Sorted List

Refer to [Appendix 7.1](#) for ADT Sorted List

- Data
 - A collection of objects in sorted order, same data type
 - The number of objects in the collection
- Operations
 - Add a new entry
 - Remove an entry
 - Check if a certain value is contained in the list
 - Clear the list
 - Return the number of entries in the list
 - Check if list is empty

Note: a sorted list will not let you add or replace an entry by position

Comparing Entries

- We need to be able to *compare entries* in order to determine the correct location to insert a new entry.
 - Thus, the objects in the sorted list must be *Comparable* i.e., must implement the method `compareTo`.
 - To enforce this requirement, we write
<T extends Comparable<T>>

Sorted Array List Implementation

- Sample code in \Chapter7\adt:
 - **SortedListInterface.java**
 - Note the generic type declaration in the interface header:
<T extends Comparable<T>>
 - **SortedArrayList.java**
 - Note the generic type declaration in the class header:
<T extends Comparable<T>>
 - Note the **new** statement to construct the array in the constructor:
list = (T[]) new Comparable[initialCapacity];
because the generic type enforces the requirement that the entries are *Comparable*.
 - **SortedArrayListDriver.java**

Sorted Linked List Implementation

- Sample code in folder \Chapter7\adt:
 - `SortedLinkedList.java`
 - `add()` method
 - If list is in ascending order, insert new entry just before first entry not smaller than new entry
 - `SortedListInterface.java`

The Method **add**

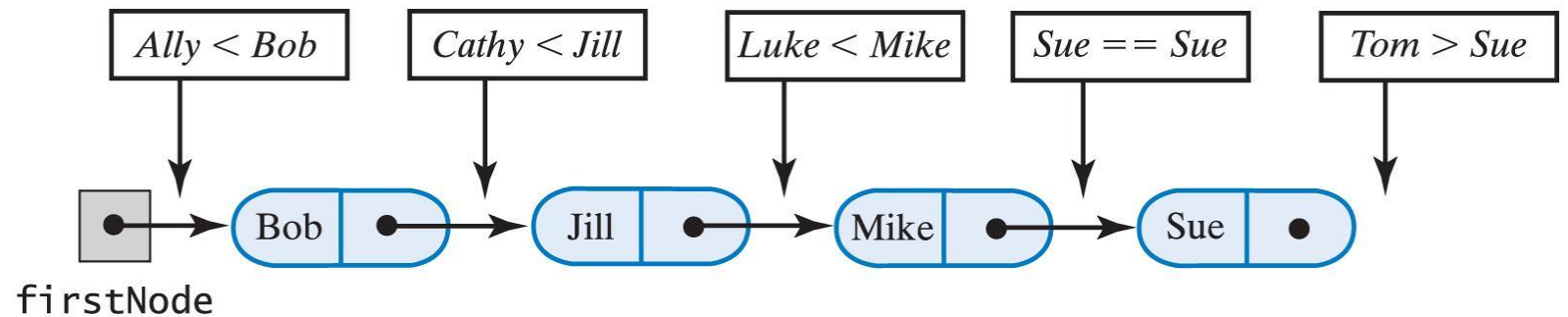


Fig. 13.1: Insertion points of names into a sorted chain of linked nodes.

The Method **add**

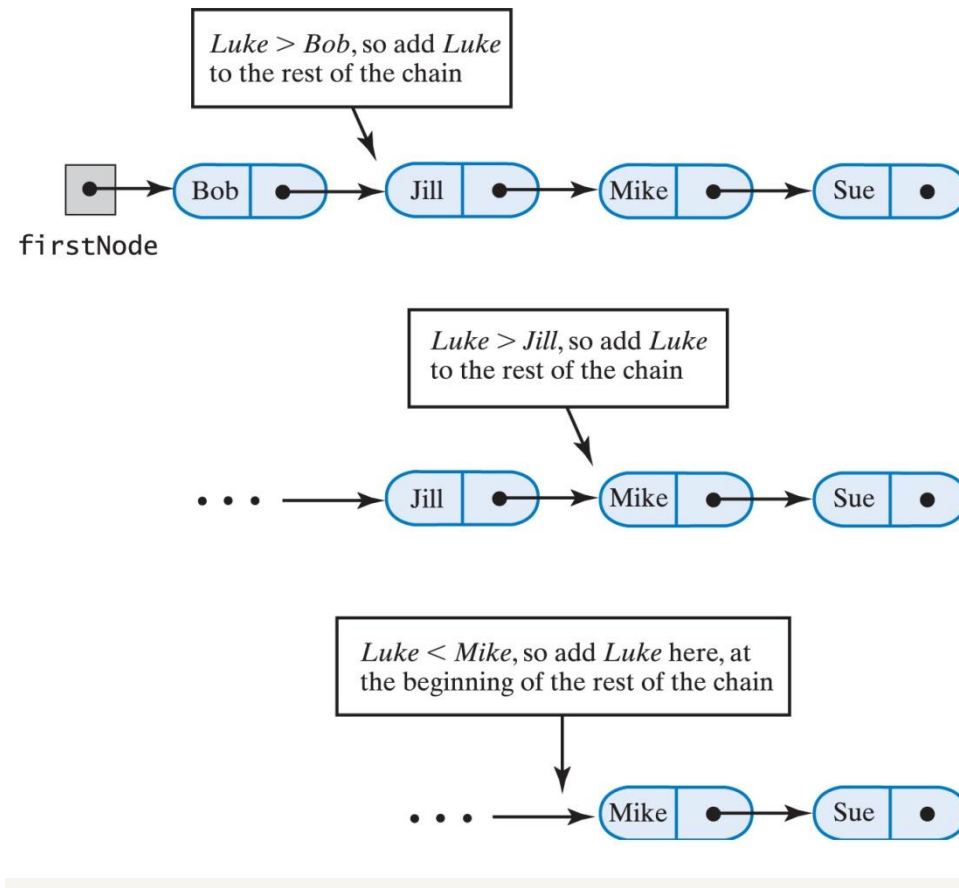


Fig. 13.2: Recursively adding *Luke* to a sorted chain of names

The Method **add**

(a) The list before any additions



(b) As `add("Ally", firstNode)` begins execution

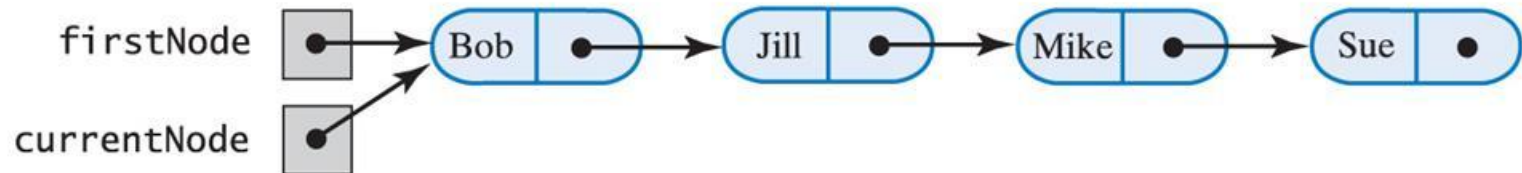
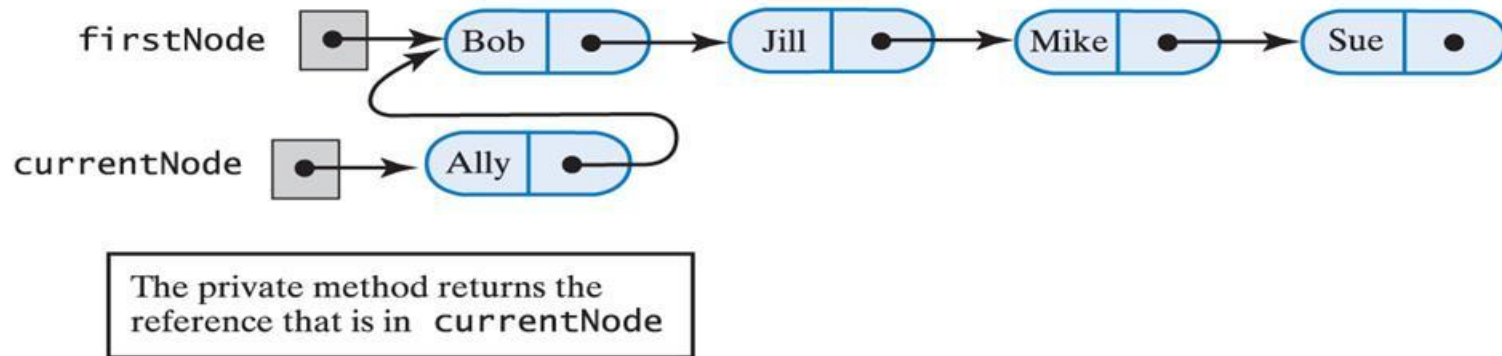


Fig. 13.3: Recursively adding *a* node at the beginning of the chain (continued →)

The Method **add**

(c) After a new node is created (the base case)



(d) After the public `add` assigns the returned reference to `firstNode`

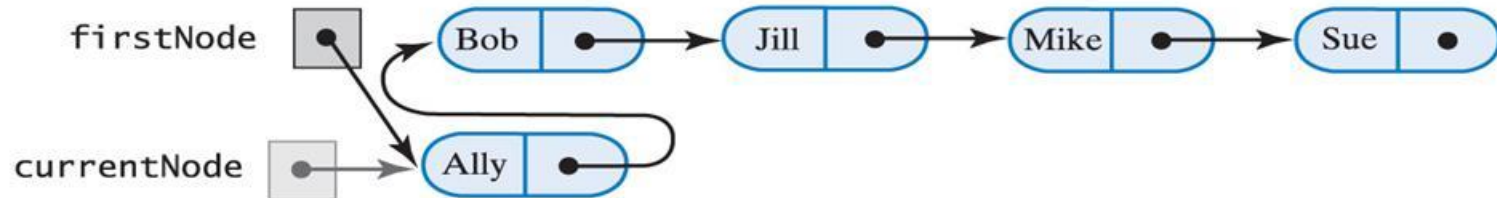
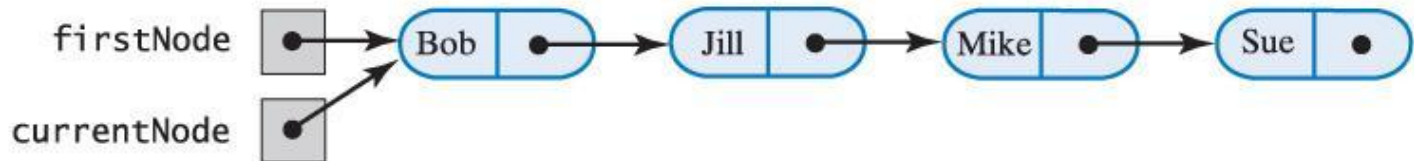


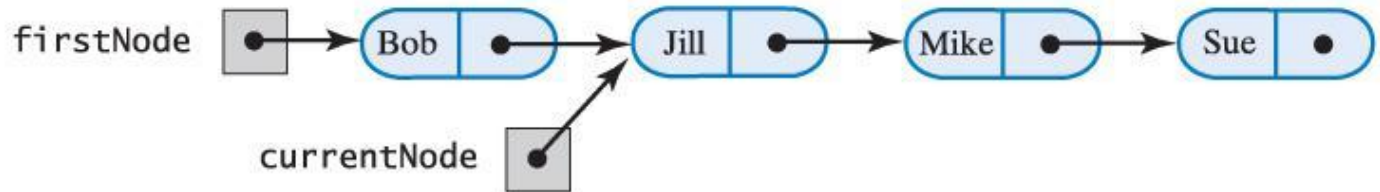
Fig. 13.3: (ctd) Recursively adding a node at the beginning of the chain.

The Method **add**

(a) As `add("Luke", firstNode)` begins execution



(b) As the recursive call `add("Luke", currentNode.getNextNode())` begins execution



(c) As the recursive call `add("Luke", currentNode.getNextNode())` begins execution

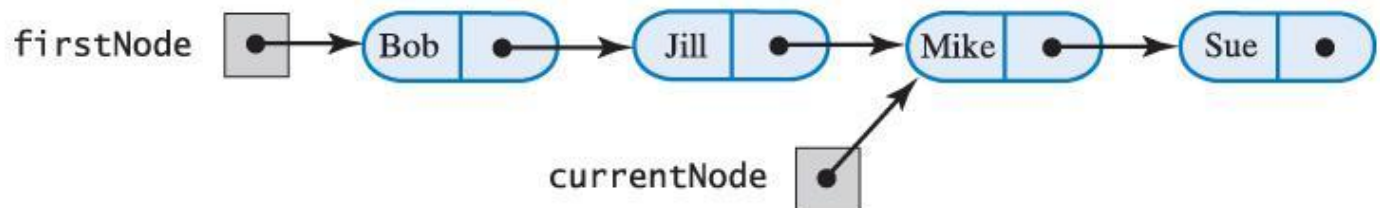
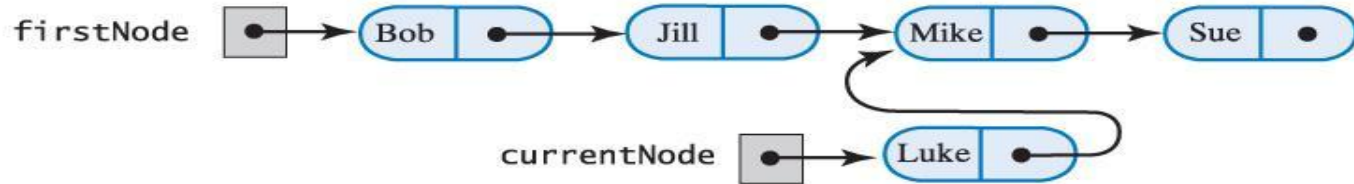


Fig. 13.4: Recursively adding a node between existing nodes in a chain (continued →)

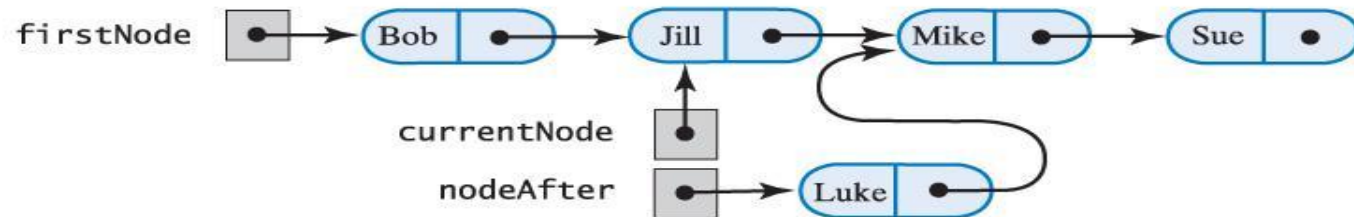
The Method **add**

(d) After a new node is created (the base case)



The private method returns the reference that is in `currentNode`

(e) After the returned reference is assigned to `nodeAfter`



(f) After `currentNode.setNextNode(nodeAfter)` executes

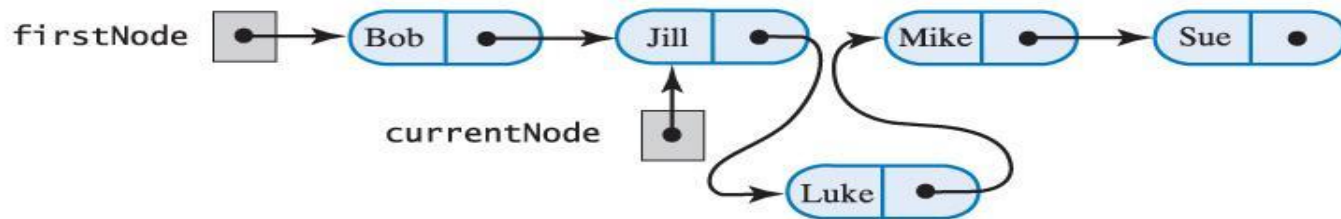


Fig. 13.4: (ctd) Recursively adding a node between existing nodes in a chain.

Efficiency of the Linked Implementation

ADT Sorted List Operation	Array	Linked
add(newEntry)	$O(n)$	$O(n)$
remove(anEntry)	$O(n)$	$O(n)$
contains(anEntry)	$O(n)$	$O(n)$
clear()	$O(1)$	$O(1)$
getNumberOfEntries()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$

Fig. 13.5: The **worst-case** efficiencies of the operations on the ADT sorted list for two implementations

Exercise 7.1



Compare and contrast how each of the following methods would be implemented in an *unsorted list* and a *sorted list* using linked implementation. Provide detailed explanations. Your answers may also include appropriate diagrams for illustration.

- (i) `void add(T newEntry)` – adds the given entry to the list. (6 marks)

Review of Learning Outcomes

You should now be able to

- Use a sorted list in a program
- Describe the differences between the ADT list and the ADT sorted list
- Implement the ADT sorted list by using an array
- Implement the ADT sorted list by using a chain of linked nodes